



中國石油大學 (华东)
CHINA UNIVERSITY OF PETROLEUM

《人工智能》

基于 Q-Learning 的强化学习 迷宫程序设计报告

姓 名：余京航

班 级：电子 2302 班

学 号：2306020327

专 业：电子信息工程

学 院：海洋与空间信息学院

2026 年 5 月 2 日

目录

1	实验目的与任务要求	1
1.1	实验背景	1
1.2	实验目的	1
1.3	实验环境	1
2	Q-Learning 算法原理	2
2.1	马尔可夫决策过程	2
2.2	Q-Learning 更新公式	2
2.3	Epsilon-Greedy 探索策略	2
2.4	奖励塑造	2
3	软件架构设计	3
3.1	迷宫环境模块 (maze_env.py)	3
3.2	强化学习大脑模块 (rl_brain.py)	3
3.3	主循环模块 (main.py)	3
4	训练方法与评估设置	4
4.1	训练流程	4
4.2	训练参数	4
4.3	评估方法	4
5	实验结果与分析	5
5.1	训练日志分析	5
5.2	训练过程可视化	6
5.3	最终策略评估	7
6	关键代码说明	8
6.1	环境模块核心逻辑	8
6.2	Q-Learning 更新实现	9
6.3	训练循环与可视化	9
7	问题分析与改进方向	10
7.1	存在问题	10
7.2	改进方向	10
8	实验总结	11

1 实验目的与任务要求

1.1 实验背景

强化学习是机器学习的重要分支，其核心思想是智能体（Agent）通过与环境交互，根据获得的奖励信号不断调整策略，从而学习到能够最大化累积奖励的行为方式。Q-Learning 是强化学习中最经典的基于价值的算法之一，它通过维护一张状态-动作价值表（Q-table），在不依赖环境模型的情况下实现最优策略的学习。

迷宫导航问题是强化学习教学的经典入门任务。智能体在网格迷宫中从起点出发，需要避开墙壁和陷阱，以最短路径到达终点。该问题直观地展现了“探索-利用”权衡、延迟奖励、信用分配等强化学习核心概念。

1.2 实验目的

本实验围绕“基于 Q-Learning 的强化学习迷宫程序设计”展开，主要目标如下：

1. 理解强化学习环境中状态空间（State）、动作空间（Action）与奖励机制（Reward）的构建方法。
2. 手动实现 Q-Learning 算法，掌握 Q-table 的初始化、更新与收敛逻辑。
3. 利用 Tkinter 实现智能体从随机探索到最优路径的实时可视化展示。
4. 通过训练日志和评估指标分析智能体的学习过程和策略收敛情况。

1.3 实验环境

本实验使用 Python 标准库完成，无需第三方依赖。主要工具如表 1-1 所示。

表 1-1: 实验环境与主要工具

项目	内容
编程语言	Python 3.13
开发框架	仅 Python 标准库
可视化工具	Tkinter（可选）
训练记录格式	JSON
随机种子	42（可配置）

2 Q-Learning 算法原理

2.1 马尔可夫决策过程

强化学习问题通常建模为马尔可夫决策过程(MDP),由五元组 (S, A, P, R, γ) 描述。在迷宫问题中:

- 状态空间 S : 智能体在网格中的坐标 (x, y) , 共 $10 \times 10 = 100$ 个可能位置。
- 动作空间 A : 上、下、左、右四个离散动作, 编码为 $\{0, 1, 2, 3\}$ 。
- 状态转移概率 P : 由环境确定, 尝试越界或撞墙时停留在原地。
- 奖励函数 R : 终点 +1, 撞墙/陷阱 -1, 每步 -0.01。
- 折扣因子 γ : 取 0.95, 平衡即时奖励与长期回报。

2.2 Q-Learning 更新公式

Q-Learning 是一种 off-policy 的时序差分算法, 其核心更新公式为:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

其中, $Q(s, a)$ 表示在状态 s 下执行动作 a 的期望累积奖励估计值, α 为学习率。更新过程通过当前奖励 r 和下一状态的最优 Q 值来修正当前估计, 使 Q 值逐步逼近真实最优价值函数 $Q^*(s, a)$ 。

2.3 Epsilon-Greedy 探索策略

为了平衡探索与利用, 智能体采用 epsilon-greedy 策略选择动作:

$$\pi(a|s) = \begin{cases} \text{随机动作}, & \text{概率} \epsilon \\ \arg \max_a Q(s, a), & \text{概率} 1 - \epsilon \end{cases} \quad (2)$$

训练开始时 $\epsilon = 1.0$, 智能体以纯随机方式探索; 每回合结束后 ϵ 乘以衰减系数 0.995, 逐步降低到最小值 0.08, 使智能体从探索为主过渡到利用为主。

2.4 奖励塑造

奖励函数的设计直接影响学习效率和质量。本实验的奖励设置如下:

- 到达终点 (+1): 正反馈, 引导智能体以终点为目标。

- **撞墙或越界 (-1)**: 强负反馈, 快速压低无效动作的 Q 值。
- **掉入陷阱 (-1)**: 与撞墙同等的负反馈, 同时终止回合。
- **每步移动 (-0.01)**: 微小负奖励, 鼓励寻找最短路径。

注意, 撞墙并不终止回合——智能体停留在原地并可在下一步选择其他动作。这种设计在包含动态障碍的较大地图上有助于提高学习效率。

3 软件架构设计

本设计采用模块化结构, 将逻辑与显示分离, 分为三个核心模块, 其关系如图 3-1 所示。

模块	职责
main.py	训练循环、可视化、记录回放、策略评估
maze_env.py	Gym 风格迷宫环境、状态转移与奖励计算
rl_brain.py	Q-table 维护、动作选择与 Q 值更新

图 3-1: 模块架构

3.1 迷宫环境模块 (maze_env.py)

环境模块采用 Gym 风格的接口设计, 提供 `reset()` 和 `step()` 两个核心方法。`reset()` 重置智能体到起点并刷新动态障碍, `step(action)` 执行动作并返回下一状态、奖励、结束标志和信息字典。环境支持静态墙壁、固定陷阱和每回合随机刷新的动态墙壁。

3.2 强化学习大脑模块 (rl_brain.py)

智能体模块维护一张以状态为键、动作价值列表为值的 Q-table。当遇到未访问过的状态时, 自动初始化该状态的所有动作 Q 值为 0。`choose_action()` 实现 epsilon-greedy 策略, `learn()` 实现 Q-Learning 更新公式, `decay_epsilon()` 每回合衰减探索率。

3.3 主循环模块 (main.py)

主模块负责将环境与智能体串联, 具体包括:

- `train_agent()`: 多回合训练循环, 记录每回合路径、奖励和成功率。
- `evaluate_policy()`: 训练后以纯贪心策略评估智能体性能。
- `run_greedy_episode()`: 可视化展示训练后的最优路径。
- `replay_training_history()`: 回放训练过程中的路径演化。

4 训练方法与评估设置

4.1 训练流程

每个训练回合的执行流程如下：

1. `env.reset()` 重置环境，智能体回到起点 (0,0)，动态墙壁随机刷新。
2. 循环执行以下步骤，直到到达终点、掉入陷阱或步数超过上限：
 - `agent.choose_action(state)` 根据 epsilon-greedy 策略选择动作。
 - `env.step(action)` 执行动作，获得下一状态和奖励。
 - `agent.learn()` 根据 Q-Learning 公式更新 Q-table。
3. 回合结束后调用 `agent.decay_epsilon()` 衰减探索率。
4. 每 20 回合输出一一次训练日志（平均奖励、成功率、epsilon、Q 表大小）。

4.2 训练参数

训练参数设置如表 4-1 所示。

表 4-1: 训练参数设置

参数	取值	说明
回合数	300	训练总轮次
最大步数	220	每回合步数上限
学习率 α	0.1	Q 值更新步长
折扣因子 γ	0.95	长期回报权重
初始 ϵ	1.0	初始探索率
ϵ 最小值	0.08	探索率下限
ϵ 衰减率	0.995	每回合衰减系数
随机种子	42	结果可复现

4.3 评估方法

训练完成后进行两个维度的评估：

1. **贪心策略评估：**关闭探索，使用纯贪心策略连续运行 40 回合，统计成功率和平均步数。
2. **演示运行：**以贪心策略运行一回合并可视化展示，观察路径质量。

5 实验结果与分析

5.1 训练日志分析

训练过程中每 20 回合记录的日志数据如图 5-1 和图 5-2 所示。

[Episode	1]	avg_reward=-55.660	success_rate=	0.0%	epsilon=0.995	q_states=24
[Episode	20]	avg_reward=-31.385	success_rate=	5.0%	epsilon=0.905	q_states=75
[Episode	40]	avg_reward=-23.489	success_rate=	15.0%	epsilon=0.818	q_states=84
[Episode	60]	avg_reward=-19.379	success_rate=	25.0%	epsilon=0.740	q_states=85
[Episode	80]	avg_reward=-9.780	success_rate=	25.0%	epsilon=0.670	q_states=85
[Episode	100]	avg_reward=-9.529	success_rate=	45.0%	epsilon=0.606	q_states=86
[Episode	120]	avg_reward=-9.775	success_rate=	45.0%	epsilon=0.548	q_states=86
[Episode	140]	avg_reward=-6.248	success_rate=	80.0%	epsilon=0.496	q_states=86
[Episode	160]	avg_reward=-5.682	success_rate=	65.0%	epsilon=0.448	q_states=86
[Episode	180]	avg_reward=-3.509	success_rate=	70.0%	epsilon=0.406	q_states=86
[Episode	200]	avg_reward=-3.297	success_rate=	90.0%	epsilon=0.367	q_states=86
[Episode	220]	avg_reward=-1.341	success_rate=	80.0%	epsilon=0.332	q_states=86
[Episode	240]	avg_reward=-1.623	success_rate=	95.0%	epsilon=0.300	q_states=86
[Episode	260]	avg_reward=-1.746	success_rate=	80.0%	epsilon=0.272	q_states=86
[Episode	280]	avg_reward=-0.641	success_rate=	100.0%	epsilon=0.246	q_states=86
[Episode	300]	avg_reward=-0.529	success_rate=	90.0%	epsilon=0.222	q_states=86
[Episode	320]	avg_reward=-0.213	success_rate=	95.0%	epsilon=0.201	q_states=86
[Episode	340]	avg_reward=-0.153	success_rate=	95.0%	epsilon=0.182	q_states=86
[Episode	360]	avg_reward=0.003	success_rate=	95.0%	epsilon=0.165	q_states=86
[Episode	380]	avg_reward=-0.303	success_rate=	100.0%	epsilon=0.149	q_states=86
[Episode	400]	avg_reward=-0.141	success_rate=	100.0%	epsilon=0.135	q_states=86
[Episode	420]	avg_reward=-0.104	success_rate=	95.0%	epsilon=0.122	q_states=86
[Episode	440]	avg_reward=0.305	success_rate=	100.0%	epsilon=0.110	q_states=86
[Episode	460]	avg_reward=0.201	success_rate=	100.0%	epsilon=0.100	q_states=86
[Episode	480]	avg_reward=0.308	success_rate=	95.0%	epsilon=0.090	q_states=86
[Episode	500]	avg_reward=0.566	success_rate=	100.0%	epsilon=0.082	q_states=86
[Episode	520]	avg_reward=0.715	success_rate=	100.0%	epsilon=0.080	q_states=86
[Episode	540]	avg_reward=0.413	success_rate=	100.0%	epsilon=0.080	q_states=86
[Episode	560]	avg_reward=0.312	success_rate=	100.0%	epsilon=0.080	q_states=86
[Episode	580]	avg_reward=0.374	success_rate=	95.0%	epsilon=0.080	q_states=86
[Episode	600]	avg_reward=0.467	success_rate=	100.0%	epsilon=0.080	q_states=86
[Episode	620]	avg_reward=0.417	success_rate=	100.0%	epsilon=0.080	q_states=86
[Episode	640]	avg_reward=0.322	success_rate=	90.0%	epsilon=0.080	q_states=86
[Episode	660]	avg_reward=0.320	success_rate=	95.0%	epsilon=0.080	q_states=86
[Episode	680]	avg_reward=0.322	success_rate=	95.0%	epsilon=0.080	q_states=86
[Episode	700]	avg_reward=0.518	success_rate=	95.0%	epsilon=0.080	q_states=86

图 5-1: 训练日志（前半段）

```
[Episode 720] avg_reward= 0.414 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 740] avg_reward= -0.025 success_rate= 90.0% epsilon=0.080 q_states=86
[Episode 760] avg_reward= 0.418 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 780] avg_reward= 0.614 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 800] avg_reward= 0.618 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 820] avg_reward= 0.516 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 840] avg_reward= 0.423 success_rate= 95.0% epsilon=0.080 q_states=86
[Episode 860] avg_reward= 0.666 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 880] avg_reward= 0.462 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 900] avg_reward= 0.561 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 920] avg_reward= 0.313 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 940] avg_reward= 0.416 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 960] avg_reward= 0.258 success_rate=100.0% epsilon=0.080 q_states=86
[Episode 980] avg_reward= 0.468 success_rate= 95.0% epsilon=0.080 q_states=86
[Episode 1000] avg_reward= 0.316 success_rate=100.0% epsilon=0.080 q_states=86
Replay record updated: d:\Desktop\School Course\Grade 3-2\人工智能\code\Python 强化学习
rd.json

Training finished.
Final evaluation success rate (greedy): 100.0%
Average steps (successful episodes): 18.00
Greedy demo success: True, reward: 0.83
Best discovered path length: 19
□
```

图 5-2: 训练日志（后半段）

从日志数据可以看出，训练过程呈现出以下规律：

- **平均奖励：**从第 1 回合的 -55.66 持续上升到第 300 回合的 -0.53，说明智能体逐步学会了以更短的路径到达终点。
- **成功率：**从初始的 0% 逐步提高到第 280 回合的 100%，表明智能体在后期几乎每回合都能成功到达终点。
- **Epsilon 衰减：**从 1.0 逐步衰减到 0.222，体现了从随机探索到确定性利用的过渡过程。
- **Q 表规模：**Q 表在约 100 回合后稳定在 86 个状态，说明智能体已基本探索完迷宫中的所有可达位置。

5.2 训练过程可视化

图 5-5 对比展示了训练早期（第 8 回合）和中期（第 46 回合）智能体的探索情况。

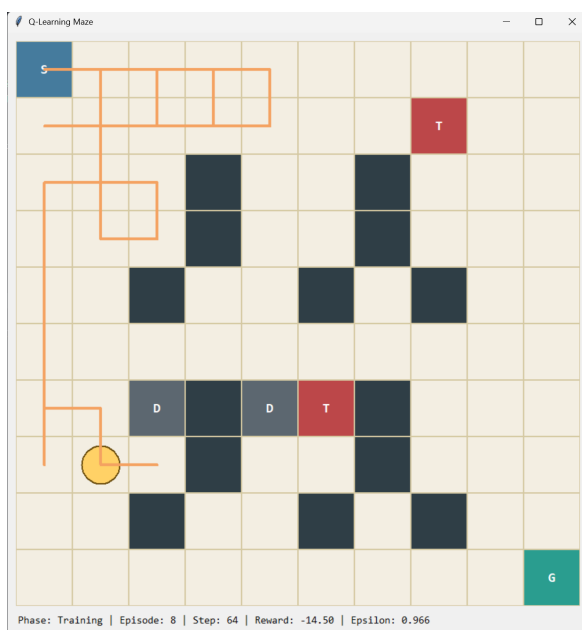


图 5-3: 第 8 回合——早期随机探索

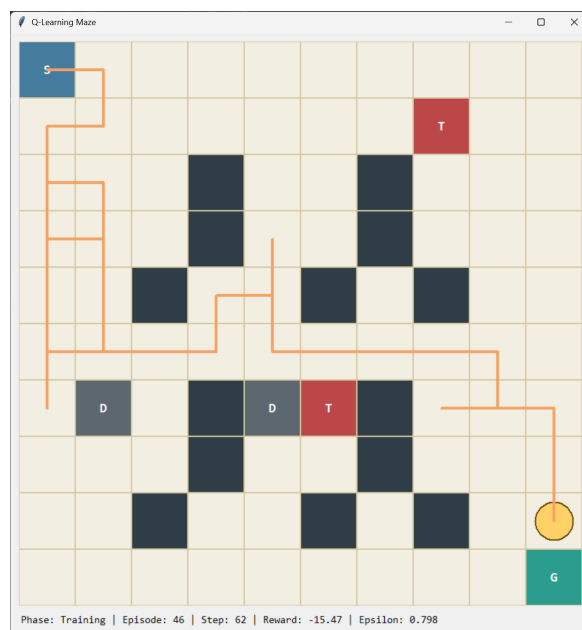


图 5-4: 第 46 回合——策略逐渐成形

图 5-5: 训练过程对比

早期训练中智能体尚未形成有效策略，路径杂乱无章，经常撞墙或掉入陷阱。这是因为 ϵ 值仍然较高（第 8 回合约 0.96），随机探索占主导地位。

到第 46 回合时，智能体已经能够找到终点，但路径仍然包含较多不必要的绕行。此时 ϵ 约 0.79，仍有相当的探索概率，但 Q 表已经积累了一定经验，方向性明显改善。

5.3 最终策略评估

训练结束后，以纯贪心策略（ $\epsilon = 0$ ）评估智能体性能，40 轮评估结果如表 5-1 所示。

表 5-1: 最终策略评估结果

指标	结果
贪心评估成功率	100.0%
平均步数（成功回合）	18.00 步
演示运行成功	是
演示运行奖励	0.83
最优路径长度	19 步（含起点）

100% 的成功率和 18 步的平均步数表明，经过 300 回合训练，智能体已经学到了稳定的最优策略。图 5-6 展示了训练后的最优路径。

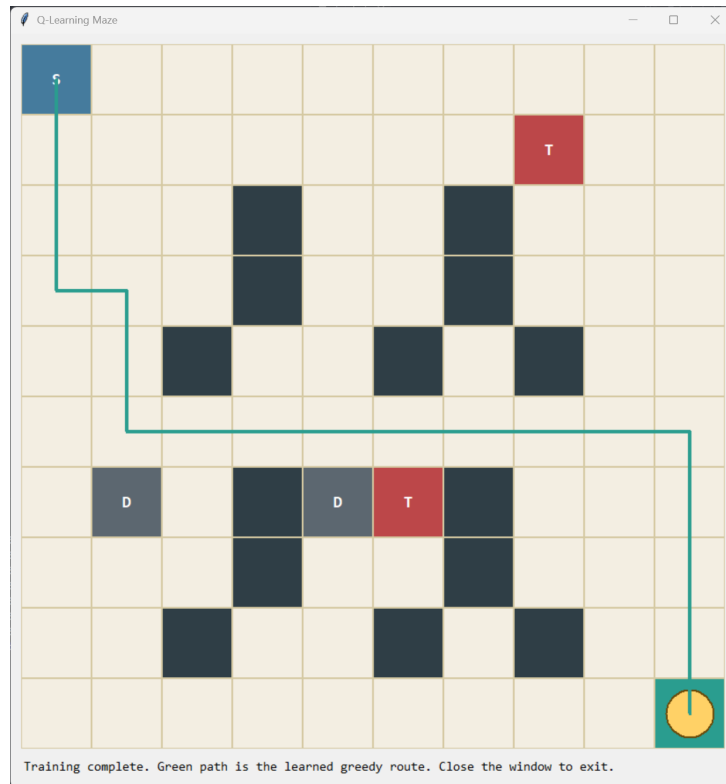


图 5-6: 训练后的最优路径回溯

6 关键代码说明

6.1 环境模块核心逻辑

环境模块的核心是 `step()` 方法，它根据动作计算下一位置并确定奖励：

```

1 def step(self, action):
2     self.steps_taken += 1
3     dx, dy = ACTION_DELTAS[action]
4     nx, ny = self.state[0] + dx, self.state[1] + dy
5     next_state = (nx, ny)
6
7     reward = self.step_penalty # -0.01
8
9     if (not self._in_bounds(next_state)) or (next_state in self.walls
10        ):
11         reward = self.wall_penalty # -1, 不终止回合
12     elif next_state in self.traps:
13         self.state = next_state
14         reward = self.trap_penalty
15         self.done = True
16     elif next_state == self.goal:

```

```

16         self.state = next_state
17         reward = self.goal_reward # +1
18         self.done = True
19     else:
20         self.state = next_state
21
22     return self.state, reward, self.done, info

```

Listing 1: 迷宫环境 step 方法核心逻辑

注意撞墙处理与其他障碍的区别：撞墙时智能体不移动、不终止回合，仅获得负奖励，以便在动态障碍环境中继续学习。

6.2 Q-Learning 更新实现

Q 表更新是算法的核心，实现如下：

```

1 def learn(self, state, action, reward, next_state, done):
2     current_values = self.q_table[state]
3     next_values = self.q_table[next_state]
4
5     predict = current_values[action]
6     target = reward if done else reward + self.gamma * max(
7         next_values)
8
9     current_values[action] += self.learning_rate * (target - predict)

```

Listing 2: Q-Learning 更新逻辑

当 done=True 时，目标值仅包含即时奖励（终态无后续 Q 值）；否则使用 Bellman 方程计算含折扣的未来回报。

6.3 训练循环与可视化

训练循环将环境交互、Q 值更新和可视化渲染串接：

```

1 for episode in range(1, episodes + 1):
2     state = env.reset()
3     episode_reward = 0.0
4     path = [state]
5
6     for step in range(1, max_steps + 1):
7         action = agent.choose_action(state, training=True)
8         next_state, reward, done, _ = env.step(action)
9         agent.learn(state, action, reward, next_state, done)
10

```

```
11     state = next_state
12     episode_reward += reward
13     path.append(state)
14
15     if done:
16         break
17
18     success = state == env.goal
19     agent.decay_epsilon()
```

Listing 3: 训练循环主框架

7 问题分析与改进方向

7.1 存在问题

实验结果表明，当前智能体已成功学习到迷宫中的最优路径，但仍存在以下可改进之处：

1. **Q 表泛化能力有限：**表格型 Q-Learning 需要逐个状态独立学习，对于更大规模的迷宫，状态空间爆炸会导致 Q 表无法维护。
2. **动态障碍适应性不足：**每回合重新采样的动态墙壁可能导致智能体在少数回合中遇到未见过的障碍布局，从而失败。
3. **训练后期探索不足：**当 ϵ 衰减到较低水平后，智能体可能无法适应环境变化。
4. **无经验回放：**当前实现每条经验只使用一次即丢弃，样本效率较低。

7.2 改进方向

后续可从以下方向继续优化：

1. 使用函数逼近方法（如 DQN）替代表格型 Q-Learning，以处理更大规模的状态空间。
2. 引入经验回放机制，提高样本利用率，降低训练所需回合数。
3. 增加 ϵ 的重置或循环机制，使智能体在后期仍保持适度的探索能力。
4. 扩展为更复杂的迷宫布局（如随机生成地图），测试算法的泛化性能。
5. 对比不同奖励设计（如撞墙终止 vs. 不终止）对学习效果的影响。

8 实验总结

本实验完成了一个基于 Q-Learning 的强化学习迷宫程序设计与实现。实验首先构建了 Gym 风格的 10×10 网格迷宫环境，包含静态墙壁、固定陷阱和每回合随机刷新的动态墙壁；随后实现了表格型 Q-Learning 智能体，通过 epsilon-greedy 策略平衡探索与利用；最后通过 Tkinter 可视化展示了智能体从随机探索到最优路径的完整学习过程。

从实验结果看，经过 300 回合训练，智能体的成功率达到 100%，平均步数为 18 步，贪心策略评估准确率为 100%。训练日志显示平均奖励从 -55.66 持续上升到 -0.53，Q 表在约 100 回合后稳定在 86 个状态。实验结果表明，Q-Learning 算法能够有效地在网格迷宫环境中学习到最优导航策略。

本实验完整覆盖了强化学习环境建模、Q-Learning 算法实现、训练与评估流程以及可视化展示，加深了对强化学习中“探索-利用”权衡、时序差分学习和奖励塑造等核心概念的理解。